

Java System Design

OOPS - class, object, interface, encapsulation, inheritance, polymorphism, abstraction

Design Patterns - Singleton, Factory, Strategy

SOLID Principles

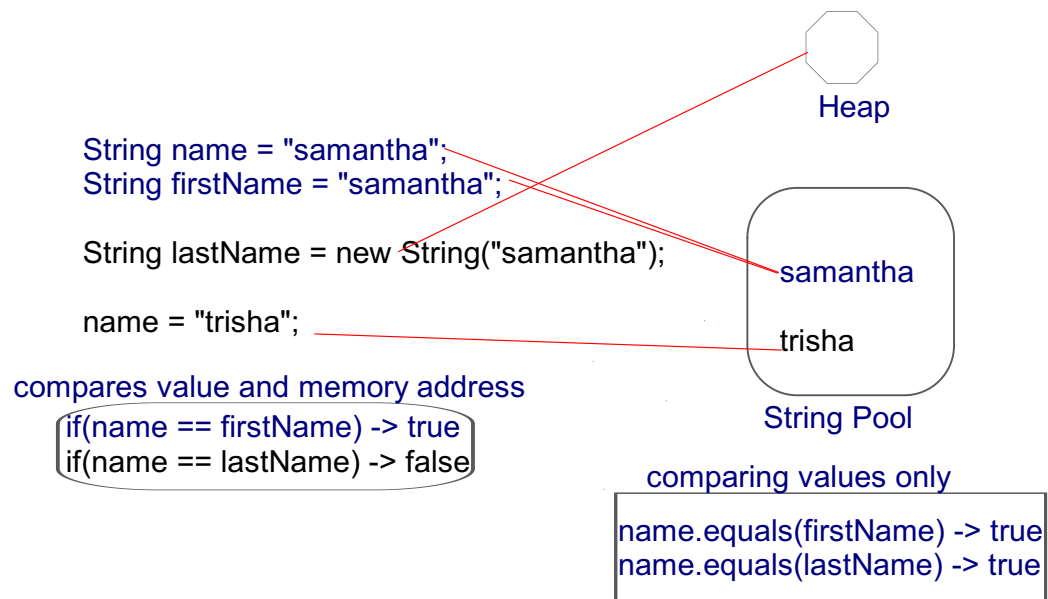
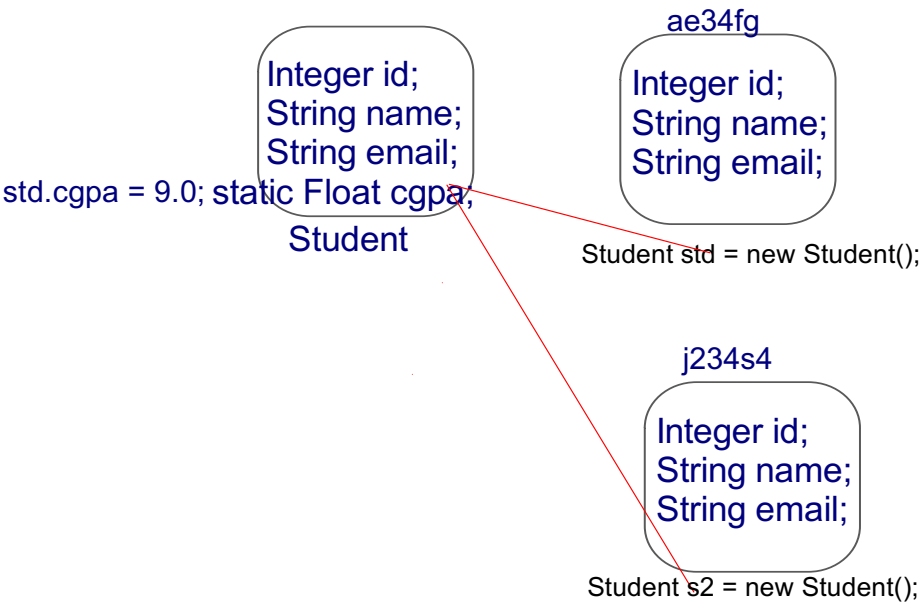
Coding Guidelines

Project Structures

Project Maintenance - DevOps Operations (GitHub)

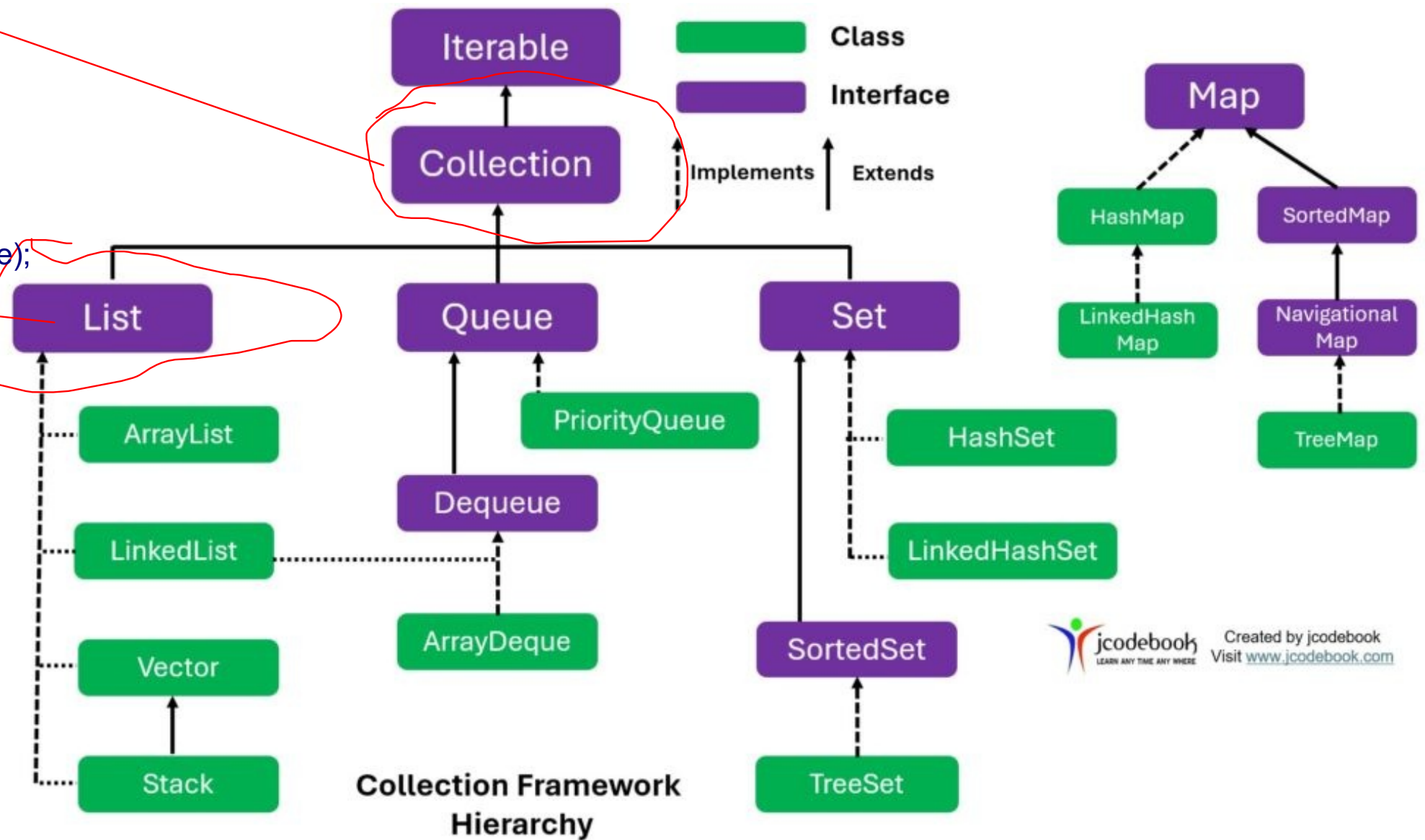
Object Oriented Programming Concepts

```
class BIT{  
    public static void main(String[] a){  
        int x = 10;  
        System.out.print(x);  
    }  
}
```



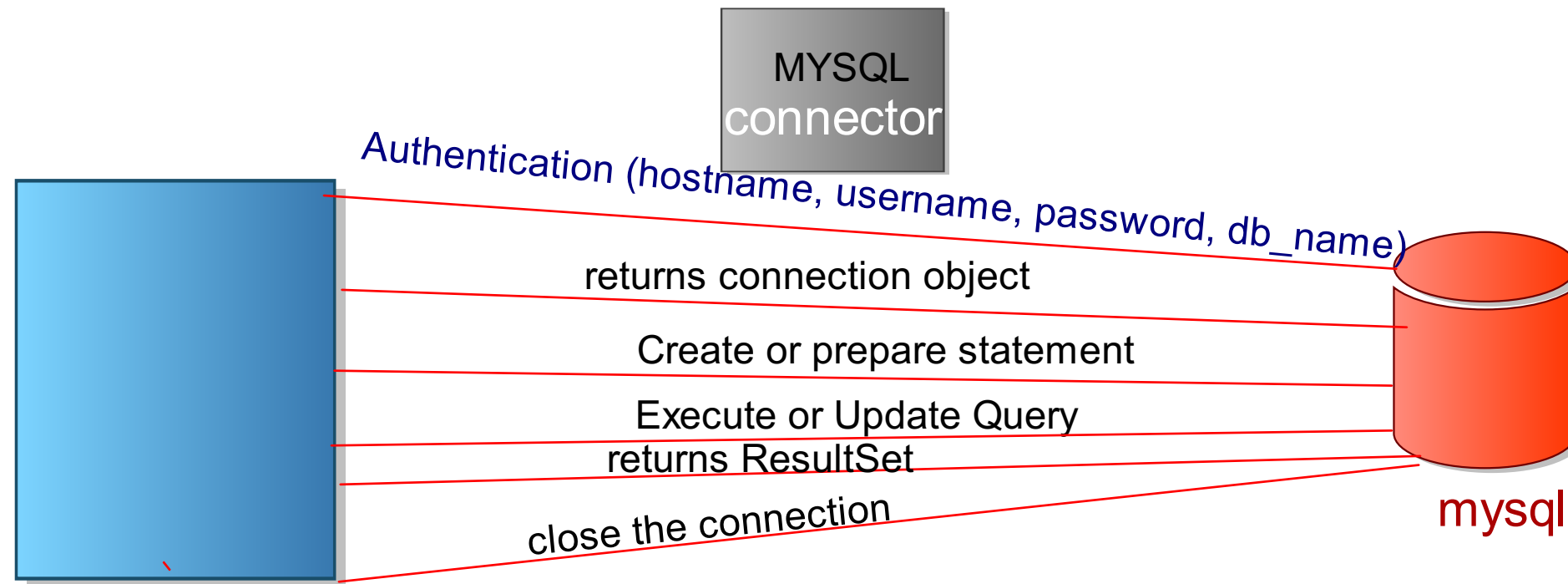
`add(Element e);`
`remove();`
`size();`
`get();`

`add(int ind, Element value);`
`get(int index);`
`addFirst(Ele e);`
`getFirst();`



Java Database Connectivity

<https://dev.mysql.com/downloads/connector/j/>



Instagram.java

```

public class Post {
    private Integer id;
    private String userName;
    private String postType;
    private String postedOn;
    private String message;

    //parameterized constructor

    //set and get methods

    //Overriden toString()
}

import java.util.List;
public interface PostService {
    public List<Post> getAllPosts();
}

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.LinkedList;
import java.util.List;
public class PostServiceImpl implements PostService {
    @Override
    public List<Post> getAllPosts() {
        List<Post> postList = new LinkedList<Post>();
        try {
            //Authentication
            Connection connection = DriverManager.getConnection
            ("jdbc:mysql://localhost:3306", "root", "Duck@!4#");
            //Statement creation
            Statement statement = connection.createStatement();
            String sql = "select id, user_name, post_type, message
                        from instagram.posts";
            //Executing query
            ResultSet response = statement.executeQuery(sql);
            while(response.next()) {
                postList.add(new Post(response.getInt("id"),
                    response.getString("user_name"),
                    response.getString("post_type"), null,
                    response.getString("message")));
            }
            //closing the connection
            connection.close();
        } catch (SQLException e) {
            System.err.println(e.getMessage());
        }
        return postList;
    }
}

```

SOLID Principles

SRP (Single Responsibility Principle)

OCP (Open/Closed Principle)

LSP (Liskov Substitution Principle)

ISP (Interface Segregation Principle)

DIP (Dependency Inversion Principle)